

Sistema de Domótica Hospitalaria

Raquel Arias Garcia, Claudia Elena Durango Cossio, Electrónica Digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	4
5. Estructura del código	5
6. Explicación de funciones principales	5
7. Comunicación	6
8. Interfaz de usuario	6
9. Procedimiento de prueba	6
10. Manejo de errores y seguridad	7

1. Identificación del proyecto

- **Nombre del proyecto:** Sistema de Domótica Hospitalaria AureCare Medical Room
- **Integrantes del equipo:** Raquel Arias Garcia, Claudia Elena Durango Cossio

2. Resumen

En este proyecto se diseñó e implementó un Sistema de Domótica Hospitalaria (SDH) utilizando un microcontrolador ESP32 programado en MicroPython. El sistema permite supervisar en tiempo real las condiciones ambientales y de seguridad de una habitación hospitalaria mediante sensores inteligentes y algoritmos de control automático.

El sistema integra un sensor DHT22 para medir temperatura y humedad relativa, un sensor MQ135 para monitorear la calidad del aire, un sensor LDR para medir el nivel de iluminación ambiental y un módulo MPU6050 que incorpora acelerómetro y giroscopio para detectar la posición y apertura de una puerta.

Además, se implementó un botón de pánico mediante interrupciones de hardware para generar alarmas inmediatas. Como elementos de actuación se utilizan ventiladores, iluminación, calefacción, buzzer de emergencia y un LED RGB para indicar visualmente el estado general del sistema.

Para la toma de decisiones se implementó una lógica difusa que evalúa simultáneamente múltiples variables ambientales y determina un nivel de riesgo. Dependiendo del resultado, el sistema clasifica la habitación en estado Normal, Alerta o Crítico, activando automáticamente los actuadores correspondientes.

Finalmente, el sistema incorpora conectividad WiFi para permitir la comunicación con un servidor central encargado de supervisar múltiples habitaciones hospitalarias.

3. Descripción del hardware

- **Microcontrolador:**

ESP32 encargado de ejecutar la lógica principal del sistema, gestionar sensores, actuadores y comunicaciones.

- **Entradas:**

- **Sensor DHT22 (pin 25):** Mide temperatura y humedad relativa del ambiente.
- **Sensor MQ135 (pin 34):** Monitorea la calidad del aire mediante la detección de gases.
- **Sensor LDR (pin 35):** Determina el nivel de iluminación ambiental.
- **MPU6050 (SDA 21 y SCL 22):** Módulo acelerómetro y giroscopio utilizado para detectar la apertura y posición de la puerta.
- **Botón de pánico (pin 18):** Permite activar una alarma crítica de forma inmediata.

■ **Salidas:**

- **Ventiladores (pines 27 y 23):** Permiten la renovación automática del aire.
- **Sistema de iluminación (pines 32 y 13):** Control automático de luz en la habitación.
- **Sistema de calefacción (pines 26 y 4):** Mantiene condiciones térmicas adecuadas.
- **Buzzer (pin 19):** Genera alarmas sonoras ante situaciones críticas.
- **LED RGB (pines 14, 12 y 15):** Indica visualmente el estado del sistema.

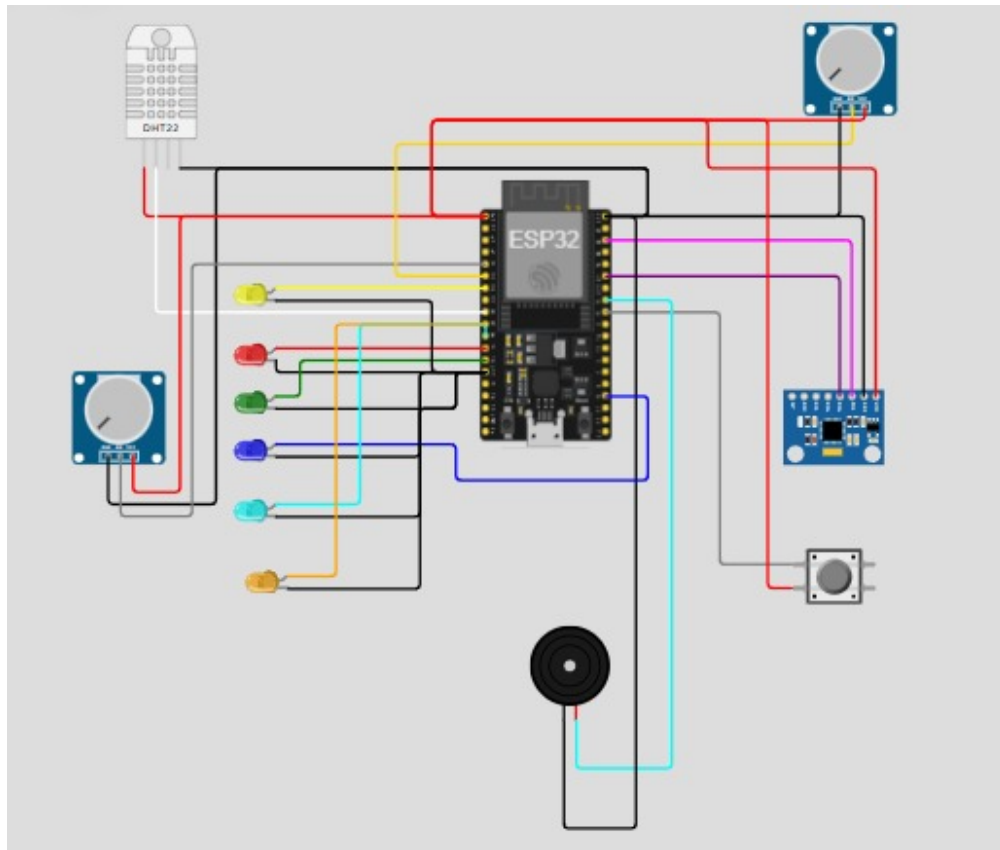
■ **Comunicación inalámbrica:**

- WiFi integrado del ESP32 para comunicación con servidor remoto.

■ **Fuente de alimentación:**

Alimentación mediante conexión USB a 5V.

Diagrama de conexión:



4. Descripción del software

- **Lenguaje:** MicroPython.

- **IDE:** Thonny.

- **Librerías utilizadas:**

- machine
- time
- dht
- network
- urequests
- struct
- json
- gc
- micropython

Flujo general del sistema:

1. Inicialización del hardware.
2. Configuración de sensores y actuadores.
3. Configuración del MPU6050 mediante I2C.
4. Conexión WiFi.
5. Lectura periódica de sensores.
6. Cálculo del ángulo de apertura de la puerta.
7. Evaluación mediante lógica difusa.
8. Clasificación del estado del sistema.
9. Activación de actuadores.
10. Actualización de indicadores visuales.
11. Comunicación con servidor remoto.
12. Recepción de comandos externos.

5. Estructura del código

- Archivo principal:

`main.py`

- Variables globales:

Temperatura, humedad, concentración de gases, iluminación, estado de puerta, aceleraciones, velocidades angulares, índice de riesgo, estado de actuadores y modo de operación.

- Funciones principales:

- `conectarwifi()`
- `leersensores()`
- `leermpu6050()`
- `controlautomatico()`
- `calcularriesgo difuso()`
- `clasificarsistema()`
- `aplicaractuadores()`
- `enviardatos servidor()`
- `recibircomandos servidor()`

6. Explicación de funciones principales

- `conectarwifi()` : *establece la conexión inalámbrica con la red disponible.*
- `leermpu6050()` : *obtiene información del acelerómetro y giroscopio para calcular el ángulo de apertura de la puerta.*
- `leersensores()` : *adquiere los datos provenientes de todos los sensores del sistema.*
- `calcularriesgo difuso()` : *implementa el sistema de inferencia difusa para determinar el nivel de riesgo.*
- `clasificarsistema()` : *clasifica el riesgo en Normal, Alerta o Crítico.*
- `controlautomatico()` : *ejecuta las decisiones automáticas del sistema.*
- `aplicaractuadores()` : *actualiza físicamente el estado de ventilación, iluminación y calefacción.*
- `enviardatos servidor()` : *transmite información al servidor central.*
- `recibircomandos servidor()` : *recibe órdenes remotas para control manual.*

7. Comunicación

La comunicación implementada en el sistema se realiza mediante:

- I2C para la comunicación entre el ESP32 y el MPU6050.
- ADC para la adquisición de señales analógicas provenientes del MQ135 y del LDR.
- WiFi para el acceso a la red.
- HTTP para la transmisión y recepción de información con el servidor central.
- Interrupciones para la detección inmediata del botón de pánico.

8. Interfaz de usuario

- LED RGB para indicar visualmente el estado del sistema.
- Buzzer para alarmas sonoras.
- Servidor remoto para supervisión y control.
- Botón de pánico para activación inmediata de emergencias.
- Consola serial para visualización de variables en tiempo real.

9. Procedimiento de prueba

1. Conectar el ESP32 al computador.
2. Abrir el IDE Thonny.
3. Cargar el archivo `main.py`.
4. Ejecutar el programa.
5. Verificar la conexión WiFi.
6. Confirmar lectura del sensor DHT22.
7. Confirmar lectura del sensor MQ135.
8. Confirmar lectura del sensor LDR.
9. Confirmar funcionamiento del MPU6050.
10. Simular apertura de puerta.
11. Verificar el cálculo del ángulo.

12. Activar el botón de pánico.
13. Confirmar el funcionamiento del buzzer.
14. Verificar el LED RGB.
15. Validar la activación automática de ventilación.
16. Validar la activación automática de iluminación.
17. Validar la activación automática de calefacción.
18. Verificar la clasificación de estados.
19. Validar la transmisión de datos al servidor.

10. Manejo de errores y seguridad

- Uso de estructuras `try-except` para evitar interrupciones del sistema.
- Gestión de errores durante la lectura de sensores.
- Manejo de errores de comunicación WiFi.
- Manejo de errores durante peticiones HTTP.
- Liberación periódica de memoria mediante `gc.collect()`.
- Uso de interrupciones para respuesta inmediata ante emergencias.
- Protección frente a errores de lectura del MPU6050.
- Activación automática de alarmas ante estados críticos.
- Posibilidad de operación automática o manual mediante comandos remotos.